INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 11

Assignment 11: Method Overriding

Assignment aim: Override inherited methods and demonstrate runtime polymorphism.

Experiment No. 11.1 — Override area() in Square

1. Experiment Title: Override area() in Square

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Method overriding replaces an inherited implementation; dynamic dispatch selects the implementation for the actual object.

5. Algorithm / Procedure:
Define area() in Shape; override it in Square to return side squared; create a square and call area(). Then compile and execute the Java program.

6. Program:

```
class Shape {
    int area() { return 0; }
}
class Square extends Shape {
    int side;
    @Override int area() { return side * side; }
}
public class Task_01_Override_area_in_Square{
    public static void main(String[] args) {
        Square s = new Square();
        s.side = 4;
        System.out.println("Area: " + s.area());
    }
}
```

7. Input: Fixed input values are declared in the program: Square side or Rectangle dimensions specified in the task.

8. Output:
Area: 16

Time / Space Complexity: O(1) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is method overriding?
- Why use @Override?
- What is the area formula for a square?

Experiment No. 11.2 — Override area() in Rectangle

1. Experiment Title: Override area() in Rectangle

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Method overriding replaces an inherited implementation; dynamic dispatch selects the implementation for the actual object.

5. Algorithm / Procedure:
Define the parent area(); override it in Rectangle using length × breadth; set dimensions and print the area.
Then compile and execute the Java program.

6. Program:

```
class Shape {
    int area() { return 0; }
}
class Rectangle extends Shape {
    int length, breadth;
    @Override int area() { return length * breadth; }
}
public class Task_02_Override_area_in_Rectangle{
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 4; r.breadth = 5;
        System.out.println("Area: " + r.area());
    }
}
```

7. Input: Fixed input values are declared in the program: Square side or Rectangle dimensions specified in the task.

8. Output:
Area: 20

Time / Space Complexity: O(1) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is the area formula for a rectangle?
- How does overriding differ from overloading?
- What is a superclass?

Experiment No. 11.3 — Use polymorphism with Shape reference

1. Experiment Title: Use polymorphism with Shape reference

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Method overriding replaces an inherited implementation; dynamic dispatch selects the implementation for the actual object.

5. Algorithm / Procedure:
Store a Square object in a Shape reference; call the overridden method; Java executes Square's implementation at runtime.
Then compile and execute the Java program.

6. Program:

```
class Shape {
    int area() { return 0; }
}
class Square extends Shape {
    int side;
    Square(int side) { this.side = side; }
    @Override int area() { return side * side; }
}
public class Task_03_Use_polymorphism_with_Shape_reference{
    public static void main(String[] args) {
        Shape s = new Square(3);
        System.out.println("Area: " + s.area());
    }
}
```

7. Input: Fixed input values are declared in the program: Square side or Rectangle dimensions specified in the task.

8. Output:
Area: 9

Time / Space Complexity: O(1) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is runtime polymorphism?
- Can a parent reference refer to a child object?
- Which area() implementation is called?